# COURSEWORK ASSIGNMENT SUBMISSION

**Name:** Daksh Srivastava **Roll No:** 150096725087
**Subject:** Software Engineering & Project Management **Day:** Day 15 — Software Design Concepts (Modularity, Cohesion, Coupling)

---

## Section 15: Classroom Activity — Design for Change

Architectural decomposition and refactoring of a tightly coupled monolithic event management codebase into high-cohesion, low-coupling modules.

### 1. Diagnosis of Existing Architectural Flaws

- **Flaw 1 (Tightly Coupled Presentation & Persistence):** UI form handlers execute direct SQL statements (`SELECT * FROM events`), meaning any database schema change directly breaks user interface code.
- **Flaw 2 (Low Cohesion):** The main controller handles user session checks, input validation, email generation, PDF rendering, and seat allocation inside a single 1,200-line function.
- **Flaw 3 (Leaky Business Rules):** Capacity checking logic is implemented in JavaScript on the browser and duplicated inside raw database triggers without a centralized business service layer.

### 2. Modular Decomposition Architecture

| Subsystem Module | Internal Cohesion Level | Primary Responsibility | Coupling Mechanism |
|---|---|---|---|
| `AuthService` | Functional (High) | User identity verification, JWT token issuance, and password hashing. | Loose (Interface / Token) |
| `CatalogService` | Functional (High) | Event retrieval, category filtering, and metadata updates. | Loose (REST / DTO) |
| `BookingEngine` | Functional (High) | Seat reservation transactions, capacity limits, and concurrency control. | Loose (Service Interface) |
| `NotificationService` | Functional (High) | Transactional email and SMS dispatch via message queues. | Decoupled (Async Queue) |
| `ReportingService` | Informational (High) | Aggregating event metrics and generating CSV/PDF export files. | Loose (Read-Only DB) |

### 3. Formal Module Interface Specification

- **Interface Method:** `BookingResult reserveSeat(UUID studentId, UUID eventId)`
- **Inputs:** `studentId` (UUID, non-null, authenticated), `eventId` (UUID, non-null, published).

- **Output Data Structure:** `BookingResult { status: ENUM [CONFIRMED, WAITLISTED, REJECTED], reservationToken: String, timestamp: Long }`
- **Handled Exceptions:** `EventFullException`, `DuplicateRegistrationException`, `ScheduleConflictException`, `SystemUnavailableException`.

## Section 18: Homework — Design Concepts Packet for Library System

Comprehensive architectural modularity and information-hiding specification for a University Library Management System.

- **1. Subsystem Decomposition:** Divided into four decoupled modules: (1) `CatalogModule` (book metadata, shelf locator); (2) `CirculationModule` (loan checkout, return, renewal rules); (3) `FineCalculationModule` (overdue policies, payment ledger); (4) `PatronModule` (student profile, active loans, eligibility).
- **2. Cohesion & Coupling Analysis:** The `FineCalculationModule` achieves high functional cohesion because every internal function relates strictly to computing penalties based on calendar day differences. It is loosely coupled to `CirculationModule` via an abstract `LoanRecordDTO`, preventing changes in physical barcode scanners from affecting penalty mathematical logic.
- **3. Information Hiding Boundaries:** (a) The internal persistence mechanism (PostgreSQL relational tables vs MongoDB documents) is completely hidden behind repository interfaces; (b) The fine calculation algorithm (holiday exemptions, grace periods) is completely encapsulated within `FineCalculator` and hidden from the checkout UI.

DARSH